Tổng hợp Kiến thức Cần Học (Ngôn ngữ Lập trình C)

(Dựa theo giáo trình)

Ngôn ngữ C bao gồm một tập hợp các khái niệm cơ bản và nâng cao được tổ chức thành các chương sau:

1. Nhập môn (Tutorial Introduction)

Phần này giới thiệu các yếu tố thiết yếu của ngôn ngữ trong các chương trình thực tế.

• **Biến và Biểu thức:** Tìm hiểu về biến, các kiểu dữ liệu như int (số nguyên) và float (số thực), và các phép tính số học.

Int, long long, float, double, char, string

Int: n <= 10^9

Long long: n <= 10^18

Float: gpa, diem,…

Double: to, de dung

Char: a, b, c, d,…

String (C++): char[]

• **Hằng số Ký hiệu:** Định nghĩa và sử dụng hằng số ký hiệu bằng #define.

#define PI 3.14

• **Nhập/Xuất Ký tự và Xử lý Văn bản:** Sử dụng các hàm getchar và putchar để đọc/ghi ký tự, và các chương trình cơ bản như sao chép tệp, đếm ký tự, đếm dòng, và đếm từ.

• **Cấu trúc Lặp và Điều khiển:** Các câu lệnh for và while để kiểm soát luồng chương trình.

Loop

For : 70%

While: 25%+++

Switch

```
Int I = 0;
for (; I < 9; I++)
for (; I < 9; ++I)
{
I = 0,…,8
}
I++;
++I;

150+;
50+;

Int I = 0;
While ( I < 9)
{
I++;
}
```

• **Mảng và Hàm:** Giới thiệu về mảng, định nghĩa hàm, đối số truyền bằng giá trị, và mảng ký tự.

```
Int a[n];
Int b[8];
Int a, b, c, d;
Int c[4]
C[0] = a;
C[1] = b;
```

B[0], b[1], …, b[1]

• **Phạm vi Biến:** Khái niệm cơ bản về biến cục bộ (automatic variables) và biến ngoại vi (external variables).

2. Kiểu Dữ liệu, Toán tử và Biểu thức (Types, Operators and Expressions)

Chương này đi sâu vào các thành phần cơ bản và các quy tắc chi phối chúng:

• **Tên Biến và Từ khóa:** Quy tắc đặt tên biến, sự khác biệt giữa chữ hoa và chữ thường, và danh sách các từ khóa dành riêng (if, else, int, float, v.v.).

• **Kiểu Dữ liệu và Kích thước:** Các kiểu cơ bản (char, int, float, double) và các bộ định tính (short, long, signed, unsigned, const, volatile, long double).

• **Hằng số:** Các loại hằng số (số nguyên, ký tự, dấu chấm động, chuỗi ký tự), bao gồm cả các chuỗi thoát (\n, \t, \a, v.v.), và hằng số liệt kê (enum).

• **Toán tử:** Toán tử số học, quan hệ, logic (&&, ||), tăng/giảm (++, --), gán, và các toán tử bitwise (&, |, ^, <<, >>, ~).

• **Chuyển đổi Kiểu:** Các quy tắc chuyển đổi kiểu tự động (implicit conversion) và ép kiểu tường minh (cast).

• **Độ ưu tiên:** Thứ tự ưu tiên và sự kết hợp của tất cả các toán tử.

3. Luồng Điều khiển (Control Flow)

Các cấu trúc giúp kiểm soát trình tự thực hiện các phép tính:

• **Câu lệnh và Khối:** Sử dụng dấu chấm phẩy (;) và dấu ngoặc nhọn {} để tạo khối (block).

• **Lựa chọn:** Các câu lệnh if-else và switch (dùng cho quyết định đa chiều dựa trên giá trị hằng số nguyên).

• **Lặp:** Các vòng lặp while, for (bao gồm vòng lặp vô hạn for (;;)) và do-while.

• **Nhảy:** Câu lệnh break (thoát khỏi vòng lặp/switch gần nhất), continue (bắt đầu vòng lặp tiếp theo), và goto (chuyển điều khiển đến nhãn trong cùng hàm).

4. Hàm và Cấu trúc Chương trình (Functions and Program Structure)

Cách tổ chức mã thành các hàm và quản lý cấu trúc chương trình lớn:

• **Cơ bản về Hàm:** Định nghĩa, khai báo prototype (kiểu mới) và giá trị trả về. Các hàm có thể trả về các kiểu dữ liệu không phải số nguyên, ví dụ double.

• **Biến Ngoại vi (External Variables):** Sử dụng biến ngoại vi để chia sẻ dữ liệu và lưu trữ giá trị vĩnh viễn (do chúng có lifetime vĩnh viễn).

• **Phạm vi và Liên kết (Scope and Linkage):** Quy tắc phạm vi của biến cục bộ và biến ngoại vi, và cách các hàm/biến được kết nối trong nhiều tệp nguồn.

• **Biến Tĩnh (Static Variables):** Sử dụng static để giới hạn phạm vi của biến ngoại vi (chỉ trong một tệp nguồn) hoặc giữ giá trị của biến nội bộ giữa các lần gọi hàm.

• **Đệ quy (Recursion):** Hàm tự gọi chính nó.

• **Bộ tiền xử lý C (C Preprocessor):** Sử dụng #include (bao gồm tệp), #define (thay thế macro với hoặc không có đối số), và biên dịch có điều kiện (#if, #ifdef, #ifndef).

5. Con trỏ và Mảng (Pointers and Arrays)

Mối quan hệ cốt lõi giữa con trỏ và mảng trong C:

• **Con trỏ và Địa chỉ:** Toán tử địa chỉ (&) và toán tử tham chiếu ngược (*).

• **Con trỏ và Đối số Hàm:** Sử dụng con trỏ để thay đổi giá trị của biến trong hàm gọi (call by reference).

• **Số học Con trỏ (Pointer Arithmetic):** Thao tác cộng/trừ con trỏ với số nguyên và phép trừ hai con trỏ cùng loại.

• **Mảng và Con trỏ:** Tên mảng là địa chỉ của phần tử đầu tiên, và mảng có thể được truy cập bằng cú pháp con trỏ (a[i] tương đương *(a+i)).

• **Mảng Con trỏ:** Mảng chứa các con trỏ (thường dùng để xử lý các chuỗi ký tự có độ dài khác nhau).

• **Đối số Dòng lệnh (Command-line Arguments):** Truy cập các đối số truyền vào chương trình qua argc và argv.

• **Con trỏ tới Hàm (Pointers to Functions):** Khai báo và sử dụng con trỏ để trỏ đến các hàm.

6. Cấu trúc, Liên hợp và Typedef (Structures, Unions and Typedef)

Các cách tổ chức dữ liệu phức tạp:

• **Cấu trúc (Structs):** Tập hợp các biến (có thể khác kiểu) được nhóm dưới một tên. Các thao tác hợp lệ bao gồm gán, sao chép, lấy địa chỉ (&), và truy cập thành viên (. hoặc -> cho con trỏ).

• **Mảng Cấu trúc:** Khai báo và khởi tạo mảng các cấu trúc (ví dụ: đếm từ khóa).

• **Cấu trúc Tự tham chiếu (Self-referential Structures):** Cấu trúc chứa con trỏ trỏ đến chính nó (ví dụ: dùng để xây dựng cây nhị phân).

• **Liên hợp (Unions):** Một biến có thể chứa các đối tượng thuộc các kiểu khác nhau tại các thời điểm khác nhau, chia sẻ cùng một vùng bộ nhớ.

• **Typedef:** Tạo tên kiểu dữ liệu mới (tên đồng nghĩa) để tăng tính di động và khả năng đọc.

• **Trường Bit (Bit-fields):** Dùng để đóng gói nhiều đối tượng nhỏ (thường là cờ/flag) vào một từ máy.

7. Nhập và Xuất (Input and Output)

Sử dụng Thư viện Chuẩn C (<stdio.h>):

• **I/O Chuẩn:** Các hàm getchar, putchar, printf (xuất có định dạng), và scanf (nhập có định dạng).

• **Truy cập Tệp (File Access):** Mở, đọc, ghi tệp bằng fopen, getc, putc, fclose.

• **I/O Trực tiếp:** Các hàm như fgets (đọc dòng) và fputs (ghi chuỗi).

• **Xử lý Lỗi I/O:** Sử dụng stderr để in thông báo lỗi và hàm exit để kết thúc chương trình với mã trạng thái.

• **Hàm Khác:** Các hàm quản lý chuỗi (<string.h>), toán học (<math.h>), và quản lý bộ nhớ (<stdlib.h>).

8. Giao diện Hệ thống UNIX (The UNIX System Interface)

Phần này giới thiệu về tương tác cấp thấp với hệ điều hành (dù các khái niệm tương tự tồn tại ở các hệ điều hành khác):

• **File Descriptors (Mô tả Tệp):** Sử dụng các số nguyên nhỏ (0, 1, 2) cho I/O chuẩn.

• **I/O Cấp thấp:** Các lệnh hệ thống read và write.

• **Mở, Tạo, Đóng Tệp:** Các lệnh open, creat, close, và unlink.

• **Truy cập Ngẫu nhiên:** Sử dụng lseek để di chuyển con trỏ tệp.

• **Ví dụ về Thư viện Chuẩn:** Cách triển khai đơn giản của fopen và getc sử dụng các lệnh hệ thống cấp thấp.

• **Cấp phát Bộ nhớ:** Triển khai một bộ cấp phát bộ nhớ chung (general-purpose storage allocator) như malloc và free sử dụng cấu trúc, liên hợp và typedef.

II. Người mới cần học và làm quen những gì

Đối với người mới bắt đầu, việc quan trọng là làm quen với các chi tiết cơ khí của lập trình C và tập trung vào các khái niệm cốt lõi:

1. Làm chủ Cơ chế Lập trình Cơ bản

• **Môi trường:** Nhanh chóng học cách **tạo văn bản chương trình, biên dịch thành công, tải và chạy chương trình** trên hệ thống của bạn.

• **Cấu trúc Chương trình C:** Hiểu rằng chương trình C bao gồm các hàm và biến, và mọi chương trình phải có hàm main.

• **Bắt đầu với I/O:** Nắm vững cách sử dụng #include <stdio.h> và hàm printf để in thông báo ra màn hình (ví dụ: chương trình "hello, world"). Đặc biệt chú ý đến **ký tự xuống dòng \n**, vì printf không tự động thêm nó.

• **Sử dụng Chú thích:** Làm quen với việc sử dụng chú thích (/* ... */) để giải thích chương trình, làm cho chương trình dễ hiểu hơn.

2. Nền tảng Ngôn ngữ (Chương 1 & 2)

• **Khai báo Biến và Kiểu dữ liệu:** Luôn luôn khai báo biến trước khi sử dụng. Hiểu sự khác biệt giữa các kiểu dữ liệu cơ bản (int, float) và cách chúng ảnh hưởng đến các phép toán (ví dụ: phép chia số nguyên sẽ cắt bỏ phần thập phân).

• **Điều khiển Luồng:** Làm quen với các cấu trúc lặp cơ bản **while** và **for**.

◦ Hãy nhớ rằng các vòng lặp này kiểm tra điều kiện ở đầu vòng lặp. Điều này đảm bảo rằng các chương trình xử lý thông minh các điều kiện biên (ví dụ: đầu vào có độ dài bằng 0).

• **Thực hành I/O Ký tự:** Hiểu rõ cách getchar trả về ký tự tiếp theo hoặc giá trị đặc biệt **EOF** (End of File), và tại sao biến chứa kết quả của getchar nên là int chứ không phải char.

3. Kỹ năng Tổ chức Mã

• **Hàm:** Tự viết các hàm nhỏ (ví dụ: hàm power trong Chương 1) để làm quen với định nghĩa hàm, khai báo prototype, và cơ chế **truyền đối số bằng giá trị** (call by value).

• **Sử dụng Hằng số Ký hiệu:** Thay thế các "số ma thuật" (magic numbers) bằng các hằng số được định nghĩa bằng #define (ví dụ: LOWER, UPPER, STEP). Việc đặt tên hằng số bằng chữ in hoa là một quy ước phổ biến.

4. Phong cách và Thói quen Tốt

• **Định dạng Mã:** Mặc dù trình biên dịch không quan tâm, người mới cần làm quen với **quy tắc thụt lề thích hợp** để làm nổi bật cấu trúc logic của chương trình (ví dụ: thụt lề các câu lệnh bên trong if, while, for).

• **Tránh Phụ thuộc vào Thứ tự Đánh giá:** Tránh viết mã dựa vào thứ tự đánh giá cụ thể của các toán tử (trừ các trường hợp được đảm bảo như &&, ||, ?: và ,), đặc biệt khi sử dụng các toán tử có tác dụng phụ (side effects) như ++ hoặc --.

5. Các Chủ đề Nâng cao để Học Sau

Sau khi nắm vững các chương cơ bản (Chương 1, 2, 3), người mới nên tiếp tục với các chủ đề nâng cao, đặc biệt là:

• **Con trỏ và Mảng (Chương 5):** Đây là các tính năng cốt lõi của C và đòi hỏi sự hiểu biết sâu sắc về bộ nhớ và địa chỉ.

• **Cấu trúc (Chương 6):** Cách hiệu quả để tổ chức dữ liệu phức tạp.

• **Thư viện Chuẩn (Chương 7):** Học cách sử dụng các hàm thư viện I/O và các hàm tiện ích khác để viết các chương trình thực tế và dễ di chuyển.

Code tham khảo

1. Nhập môn (Tutorial Introduction)

Ví dụ về Biến và Biểu thức

```
#include <stdio.h>

int a = 5, b = 3;

int main() {

    int a = 5, b = 3;

    float c = 4.5;
```

```
    // Biểu thức tính tổng
    float result = a + b + c;

    printf("Kết quả của phép cộng là: %.2f\n", result);
    return 0;
}
```

Giải thích: Chương trình này khai báo các biến a, b là kiểu int và c là kiểu float, sau đó thực hiện phép cộng giữa chúng và in kết quả ra màn hình.

2. Kiểu Dữ liệu, Toán tử và Biểu thức

Ví dụ về Toán tử

```
#include <stdio.h>

int main() {
    int x = 10, y = 4;

    // Toán tử số học
    printf("Tổng: %d\n", x + y);
    printf("Hiệu: %d\n", x - y);
    printf("Tích: %d\n", x * y);
    printf("Chia: %d\n", x / y);
    printf("Chia dư: %d\n", x % y);

    // Toán tử tăng/giảm
    printf("Giá trị x sau khi tăng: %d\n", ++x);
    printf("Giá trị y sau khi giảm: %d\n", --y);

    return 0;
}
```

Giải thích: Chương trình này thực hiện các phép toán số học cơ bản và sử dụng toán tử tăng/giảm để thay đổi giá trị của x và y.

3. Luồng Điều khiển

Ví dụ về Câu lệnh Điều kiện (if-else)

```
#include <stdio.h>

int main() {
    int num;

    printf("Nhập một số nguyên: ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Số dương\n");
    } else if (num < 0) {
        printf("Số âm\n");
    } else {
        printf("Số không\n");
    }

    return 0;
}
```

Giải thích: Chương trình yêu cầu người dùng nhập một số và sử dụng câu lệnh if-else để kiểm tra xem số đó là dương, âm hay bằng 0.

Ví dụ về Cấu trúc lặp (for)

```
#include <stdio.h>

int main() {
```

```
    for (int i = 1; i <= 5; i++) {
        printf("Số: %d\n", i);
    }
    return 0;
}
```

Giải thích: Đây là ví dụ về vòng lặp for, chương trình in ra các số từ 1 đến 5.

4. Hàm và Cấu trúc Chương trình

Ví dụ về Hàm

```
#include <stdio.h>

// Định nghĩa hàm tính bình phương của số
int square(int num) {
    return num * num;
}

int main() {
    int number = 4;
    int result = square(number);  // Gọi hàm square
    printf("Bình phương của %d là: %d\n", number, result);
    return 0;
}
```

Giải thích: Chương trình này định nghĩa một hàm square nhận vào một số nguyên và trả về bình phương của nó. Hàm này được gọi trong hàm main để tính và in kết quả.

5. Con trỏ và Mảng

Ví dụ về Con trỏ

```
#include <stdio.h>

int main() {
```

```
    int x = 10;
    int *ptr = &x;  // Khai báo con trỏ và gán địa chỉ của x

    printf("Giá trị của x: %d\n", *ptr);  // Lấy giá trị thông qua con trỏ
    printf("Địa chỉ của x: %p\n", ptr);  // In địa chỉ của x

    return 0;
}
```

Giải thích: Chương trình này sử dụng con trỏ ptr để lưu trữ địa chỉ của biến x và sau đó truy cập giá trị của x thông qua con trỏ.

Ví dụ về Mảng và Con trỏ

```
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int *ptr = arr;  // Con trỏ trỏ đến mảng arr

    for (int i = 0; i < 5; i++) {
        printf("Phần tử thứ %d trong mảng: %d\n", i+1, *(ptr + i));
    }

    return 0;
}
```

Giải thích: Mảng và con trỏ liên quan chặt chẽ trong C. Tên mảng arr chính là địa chỉ của phần tử đầu tiên. Con trỏ ptr trỏ tới mảng và thông qua phép toán con trỏ (*(ptr + i)), chúng ta có thể truy cập các phần tử của mảng.

6. Cấu trúc, Liên hợp và Typedef

Ví dụ về Cấu trúc (Struct)

```
#include <stdio.h>

// Định nghĩa cấu trúc Sinh viên
struct Student {
    int id;
    char name[50];
    float grade;
};

int main() {
    struct Student student1 = {1, "Nguyen Van A", 8.5};

    printf("ID: %d\n", student1.id);
    printf("Tên: %s\n", student1.name);
    printf("Điểm: %.2f\n", student1.grade);

    return 0;
}
```

Giải thích: Cấu trúc Student nhóm các thông tin liên quan đến sinh viên, bao gồm id, name và grade. Chúng ta có thể tạo và sử dụng cấu trúc này trong chương trình.

Ví dụ về Typedef

```
#include <stdio.h>

// Định nghĩa tên mới cho kiểu struct Student
typedef struct {
    int id;
    char name[50];
    float grade;
```

```
} Student;

int main() {
    Student student1 = {2, "Nguyen Thi B", 9.0};

    printf("ID: %d\n", student1.id);
    printf("Tên: %s\n", student1.name);
    printf("Điểm: %.2f\n", student1.grade);

    return 0;
}
```

Giải thích: typedef cho phép chúng ta đặt tên mới cho một kiểu dữ liệu đã có. Trong ví dụ này, Student trở thành một tên mới thay cho struct ban đầu.

7. Nhập và Xuất

Ví dụ về Đọc và ghi tệp

```
#include <stdio.h>

int main() {
    FILE *file = fopen("example.txt", "w");  // Mở tệp để ghi

    if (file) {
        fprintf(file, "Xin chào, thế giới!\n");
        fclose(file);  // Đóng tệp sau khi ghi xong
        printf("Dữ liệu đã được ghi vào tệp.\n");
    } else {
        printf("Không thể mở tệp.\n");
    }
```

```
    return 0;
}
```

Giải thích: Chương trình này mở tệp example.txt để ghi, sau đó ghi một dòng văn bản vào tệp và đóng tệp lại.

8. Giao diện Hệ thống UNIX

Ví dụ về I/O Cấp thấp

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main() {
    int file = open("example.txt", O_WRONLY | O_CREAT, 0644);  // Mở tệp

    if (file != -1) {
        write(file, "Xin chào, thế giới!\n", 20);  // Ghi vào tệp
        close(file);  // Đóng tệp
        printf("Dữ liệu đã được ghi vào tệp.\n");
    } else {
        printf("Không thể mở tệp.\n");
    }

    return 0;
}
```

Giải thích: Chương trình sử dụng các hàm hệ thống Unix open, write và close để ghi dữ liệu vào tệp example.txt.